Security Test Report

FalconStrix – Stored Cross-Site Scripting (XSS) Assessment

Report ID	FALSTRX-2026-007
Component Tested	Manual alert message field, and every location that message is later displayed
Vulnerability Class Tested	Stored Cross-Site Scripting – CWE-79
Result	No vulnerability found – payload consistently escaped on output
Discovered By	Manual black-box testing (Burp Suite + direct UI inspection)
Environment	Isolated local lab – Windows 11 target, Kali Linux (VMware) attacker
Assessment Date	27–28 July 2026
Status	Closed – No Action Required

1. Summary

Testing was performed to determine whether the manual alert message field is vulnerable to Stored Cross-Site Scripting — specifically, whether a script payload submitted as an alert message would later execute in the browser of any user viewing that alert, rather than being displayed as inert text. A classic script-tag payload was submitted and traced across every surface in the application that subsequently displays alert message content. In all locations checked, the payload was rendered as literal, non-executing text. No stored XSS was identified.

2. Methodology

A single, distinctive payload was used so it could be reliably traced across every surface of the application without ambiguity:

```
<script>alert('XSS-TEST')</script>
```

This payload was submitted once via the “Create Real Alert” feature, and the resulting alert was then followed through every part of the UI that surfaces alert data, checking in each case whether a JavaScript alert box actually fired (indicating real execution) or whether the literal characters were shown as visible page text (indicating safe output escaping).

3. Evidence

3.1 Storage layer (Burp, raw API response)

```
POST /api/alerts/manual HTTP/1.1
{"severity":"MEDIUM","message":"<script>alert('XSS-TEST')</script>"}
```

```
--- Response ---
HTTP/1.1 200 OK
{"alert_id":276, "ok":true, "snapshot": {"active_alerts": [
{"message": "<script>alert('XSS-TEST')</script>", ...}
]}}
```

The payload is stored exactly as submitted, with no server-side stripping or modification — confirming any protection observed occurs at display (output) time, not at input time.

3.2 Live Alerts page

The “Description” column displayed the literal text `<script>alert('XSS-TEST')</script>` as plain, visible characters. No JavaScript alert box fired.

3.3 User Activity panel

The activity log entry “User 'zaid' created manual HIGH alert: `<script>alert('XSS-TEST')</script>`” rendered as plain text. No execution.

3.4 Incident Summary panel

The corresponding incident summary entry likewise displayed the payload as plain text with no execution.

3.5 Resolved Cases page

After resolving the alert, the case appeared in the Solved Cases table. The trigger-event column correctly shows the event type (`MANUAL_ALERT`) rather than the raw message in this particular column, and no execution occurred anywhere on this page while the record was present and inspected.

4. Summary Table

Surface	Payload Rendered As	Executed?
Live Alerts (Description column)	Literal text	No
User Activity panel	Literal text	No
Incident Summary panel	Literal text	No
Resolved Cases page	N/A (message not shown in this column)	No
Raw API / storage layer	Stored unmodified (escaping occurs on output only)	N/A

5. Analysis & Conclusion

Stored XSS requires that user-supplied content be inserted into a page's HTML without proper output encoding, allowing the browser to interpret it as markup rather than text. Across every surface tested, FalconStrix consistently HTML-escapes special characters (<, >, etc.) at render time, regardless of where the content is displayed, even though the raw value is stored unmodified in the database. This is the correct, standard defensive pattern (store raw, escape on output) and it was found to be applied consistently rather than in only one location.

Conclusion: No Stored XSS vulnerability identified via the manual alert message field across the surfaces tested in this assessment.

6. Scope Note & Follow-Up

During this test, a separate, non-security data-completeness issue was observed: alerts created via the manual alert feature do not populate resolution detail fields (resolved timestamp, process name, PID) in the Solved Cases table after being resolved, unlike alerts originating from simulated system events. This is a functional/data bug, not a security vulnerability, and is tracked separately for remediation outside the scope of this assessment. PDF report generation was also observed to be non-functional at the time of testing and is likewise tracked separately.

7. Value of This Test

As with the IDOR and SQL Injection assessments on this application, a well-documented negative result demonstrates that this vulnerability class was actively tested across multiple real display surfaces, not assumed safe or left unchecked.

8. Testing Environment

- Target: FalconStrix on Windows 11, Flask bound to 0.0.0.0:5001, isolated VMware network.
- Attacker: Kali Linux (VMware), Burp Suite Community Edition.
- All testing performed exclusively against the researcher's own locally-hosted project.
